Doomscroll

Use Case Model — Lab 5 (Updated)

Version 2.0 -- supersedes Lab 1's Initial Use Case Model (v1.0) and Lab 2's refined Use Case Model (v1.1)
Prepared by: Project Team -- SC2006
Date: 6 November 2026

Revision History

Version	Date	Author	Reason for Changes
1.0	6 September 2026	Project Team	Initial Use Case Model (Lab 1): 11 use cases covering camera access, calibration, gaze estimation, screen-content analysis, scrolling, and user control.
1.1	20 September 2026	Project Team	Reviewed for consistency with the Lab 2 conceptual model (class_diagram.jpg) and dialog map (dialog_map.jpg); no substantive flow changes.
2.0	6 November 2026	Project Team	Lab 5 update: rewritten to match the current implementation. Several use cases described flows the codebase never had (automatic confidence-based recalibration, manual-scroll suppression, configurable scroll settings) and have been corrected,

			merged, or replaced; new use cases cover the CNN Model's separate Calibration and training pipeline, which Labs 1-2 did not model. See srs.docx's “Changes Since Lab 2” section for the full list.

Actors

User (primary): the person reading on-screen content whose head and gaze are tracked, and who starts, calibrates, and controls the application from its launcher.

Webcam (secondary): the device's camera hardware, supplying the images the system analyses.

Display / OS (secondary): the operating system's window manager and scroll functions — the source of the Focused Window's bounds and on-screen text content, and the target of Autoscroll's scroll events.

Use Case Diagram

See use_case_diagram.jpg. Use Case IDs below are grouped hierarchically by functional area: 1.x Camera Access, 2.x Calibration, 3.x Gaze Tracking, 4.x Content-Aware Scroll Trigger, 5.x Scrolling and Session Control, and 6.x Application Control. «include» relationships are listed in each use case's Includes field.

Use Case Descriptions

Each use case below is documented using the Use Case Template.

Use Case 1.1 — Access Webcam

Use Case 2.1 — Quick-Start Calibration (Math Model)

Use Case 2.2 — Record, Train & Calibrate (CNN Model)

Use Case 2.3 — Calibrate Only (CNN Model)

Use Case 2.4 — Review Calibration Checkpoint

Use Case 2.5 — Clear All Calibrations

Use Case 3.1 — Estimate Gaze Point

Use Case 4.1 — Scan Screen Content

Use Case 4.2 — Evaluate Scroll Trigger

Use Case 5.1 — Autoscroll Page

Use Case 5.2 — Pause Autoscroll on Look-Away

Use Case 5.3 — End Autoscroll Session

Use Case 6.1 — Configure Appearance

Use Case 1.1 — Access Webcam

Use Case ID:	1.1		
Use Case Name:	Access Webcam		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		The system opens the device's webcam through the operating system's own camera framework at the start of a tracking-dependent step (Calibration, Recording, or Autoscroll). The system displays no explanation of its own; the operating system's native permission prompt is what the User actually responds to, the first time this happens.	
Preconditions:		1. A flow that needs the webcam (Quick Start, Record/train/calibrate, Calibrate only) has been started from the Main Page.	
Postconditions:		1. The webcam is open and streaming frames to the step that requested it, or 2. The step has stopped and reported failure in the Log (see Alternative Flows).	
Priority:		High	
Frequency of Use:		Once per step that needs the webcam; the operating system may re-prompt for permission only per its own policy, not this application's.	
Flow of Events:		1. The User starts a flow that needs the webcam from the Main Page. 2. The step opens camera index 0 via the platform capture backend (AVFoundation on macOS, DirectShow on Windows). 3. If this is the first time the application has requested the camera, the operating system shows its own permission prompt; the User allows or denies access. 4. If access is allowed and a camera is present, frames begin streaming to the step.	
Alternative Flows:		1.1.AC.1: If the webcam cannot be opened for any reason -- camera permission denied, no camera present, or the camera already in use by another application -- the step raises an error, which the launcher reports as a failed step in the Log, and the Main Page's buttons are re-enabled (Use Case 6.1's flows do not proceed).	

Exceptions:	None.
Includes:	None.
Special Requirements:	Requirement 3.1.1; Requirement 3.1.2.
Assumptions:	The User's device has a working webcam and an operating system capable of presenting a camera-permission prompt.
Notes and Issues:	Unlike Lab 1/2's version of this use case, the system has no custom in-app explanation step and cannot distinguish “permission denied” from other camera-open failures; all are reported the same way.

Use Case 2.1 — Quick-Start Calibration (Math Model)

Use Case ID:	2.1		
Use Case Name:	Quick-Start Calibration (Math Model)		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		The User completes a guided Calibration sequence using the Math Model's Quick Start path, so the system can fit a Calibration Profile (GazeCalibrator) mapping their head and eye movements to screen positions, without any CNN training involved.	
Preconditions:		1. Camera access has been granted (Use Case 1.1).	
Postconditions:		1. A Math Model Calibration Profile has been fitted and (if its validation error looks acceptable to the User) is in use for Autoscroll.	
Priority:		High	
Frequency of Use:		Whenever the User chooses Quick Start from the Main Page.	
Flow of Events:		1. The User chooses a calibration protocol: Trajectory (a single dot sweeping the whole screen on its own, ~40 seconds, the default) or Dots (a 16-point grid, fixated one point at a time). 2. The User optionally enables “Pool with recent sessions”. 3. The system runs the chosen protocol, followed by a short head-pose-variation pass (lean left/right, lean closer/sit back, tilt chin up/down). 4. The system fits a new Math Model	

	Calibration Profile from the samples captured, and, if pooling was enabled, also fits a pooled version and keeps whichever validates with the lower mean pixel error. 5. The system prints the validation error(s) to the Log. 6. The system carries out Use Case 2.4 (Review Calibration Checkpoint). 7. If the User chose to proceed, the system carries out Use Case 5.1 (Autoscroll Page) using the Math Model.
Alternative Flows:	2.1.AC.1: The User may press ESC during the protocol or the head-pose pass to abort back to the Main Page without fitting a profile.
Exceptions:	None.
Includes:	Use Case 1.1 (Access Webcam); Use Case 2.4 (Review Calibration Checkpoint); Use Case 5.1 (Autoscroll Page).
Special Requirements:	Requirement 3.2.4.1; Requirement 3.3.1; Requirement 3.3.2.
Assumptions:	The User is seated at a normal viewing distance from the screen during Calibration.
Notes and Issues:	Pooling only ever combines sessions of the same screen size and protocol; it is opt-in, not default, because a changed physical setup can make a pooled fit worse rather than better.

Use Case 2.2 — Record, Train & Calibrate (CNN Model)

Use Case ID:	2.2		
Use Case Name:	Record, Train & Calibrate (CNN Model)		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		The User records a long video of themselves following a moving cursor, which is folded into the permanent Training Dataset and used to retrain the CNN Model and re-estimate its cursor Lag Estimate, before a short Quick Calibrate round fits a per-user correction on top of it.	
Preconditions:		1. Camera access has been granted (Use Case 1.1). 2. The User has entered a Recording duration (seconds).	

Postconditions:	1. The Training Dataset has been extended with this Recording Session, the CNN Model has been retrained on it, and (if the User proceeds past the checkpoint) a per-user correction fitted for it is in use for Autoscroll.
Priority:	High
Frequency of Use:	Whenever the User chooses “Record, train & calibrate”; expected to be infrequent, since each run retrains the CNN on the whole accumulated dataset.
Flow of Events:	1. The User enters a Recording duration and starts the flow. 2. The system records a Recording Session: the User follows a cursor moving around the screen for the chosen duration, identified by the machine's own Participant ID (Requirement 3.6.5), never a real name. 3. The system extracts face crops, then feature vectors, from the raw recording into the Training Dataset, then deletes the raw recording. 4. The system re-estimates the CNN Model's Lag Estimate from the full Training Dataset, on a best-effort basis. 5. The system retrains the CNN Model on the full Training Dataset using the current Lag Estimate. 6. The system runs a fixed 9-point Quick Calibrate grid (~15 seconds) to fit a per-user correction on top of the freshly retrained CNN Model. 7. The system carries out Use Case 2.4 (Review Calibration Checkpoint). 8. If the User chose to proceed, the system carries out Use Case 5.1 (Autoscroll Page) using the CNN Model.
Alternative Flows:	2.2.AC.1: If the recording itself fails (Use Case 1.1's Alternative Flow), the system stops here and the rest of this flow (steps 3 onward) does not run.
Exceptions:	2.2.EX.1: If the system cannot determine this recording's own session identifier from its output, it leaves the raw recording in place rather than deleting the wrong folder, and continues with the rest of the flow regardless. 2.2.EX.2: If re-estimating the Lag Estimate

	(step 4) fails for any reason, the system continues to step 5 using whichever Lag Estimate was already stored.
Includes:	Use Case 1.1 (Access Webcam); Use Case 2.4 (Review Calibration Checkpoint); Use Case 5.1 (Autoscroll Page).
Special Requirements:	Requirement 3.2.4.2; Requirement 3.3.3; Requirement 3.6.5; Requirement 3.7.1; Requirement 3.7.2; Requirement 3.7.3; Requirement 3.7.4.
Assumptions:	The torch package (the “cnn” optional dependency) is installed; if it is not, the Main Page disables this flow's button entirely rather than letting it fail partway through.
Notes and Issues:	This is a specialisation of the same underlying Calibration goal as Use Case 2.1, for the CNN Model instead of the Math Model; the two are otherwise independent pipelines with no shared Calibration Profile.

Use Case 2.3 — Calibrate Only (CNN Model)

Use Case ID:	2.3		
Use Case Name:	Calibrate Only (CNN Model)		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		The User fits a new per-user correction on top of whichever CNN Model is already trained, using only the short 9-point Quick Calibrate grid, skipping the long Recording Session and retrain.	
Preconditions:		1. A trained CNN Model already exists (models/gaze_cnn.pt) -- produced by at least one prior run of Use Case 2.2.	
Postconditions:		1. A new per-user correction has been fitted for the existing CNN Model and (if the User proceeds) is in use for Autoscroll.	
Priority:		Medium	
Frequency of Use:		Whenever the User wants to re-fit their own correction without re-recording -- for example after a seating or lighting change.	
Flow of Events:		1. The User chooses “Calibrate only (CNN)” from the Main Page. 2. The system warns that accuracy will likely be worse than a full Record, train & calibrate,	

	since it only fits a correction on top of the existing model; the User confirms they want to continue. 3. The system runs the fixed 9-point Quick Calibrate grid (~15 seconds) and fits a new per-user correction. 4. The system carries out Use Case 2.4 (Review Calibration Checkpoint). 5. If the User chose to proceed, the system carries out Use Case 5.1 (Autoscroll Page) using the CNN Model.
Alternative Flows:	2.3.AC.1: If the User declines the accuracy warning in step 2, the system returns to the Main Page without running Quick Calibrate.
Exceptions:	2.3.EX.1: If no trained CNN Model exists yet, the system shows an error explaining that Record, train & calibrate must be run first, and does not proceed.
Includes:	Use Case 1.1 (Access Webcam); Use Case 2.4 (Review Calibration Checkpoint); Use Case 5.1 (Autoscroll Page).
Special Requirements:	Requirement 3.2.4.2; Requirement 3.3.3.
Assumptions:	None.
Notes and Issues:	None.

Use Case 2.4 — Review Calibration Checkpoint

Use Case ID:	2.4		
Use Case Name:	Review Calibration Checkpoint		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		Immediately after any Calibration or Quick Calibrate run finishes, the system asks the User to judge the result (based on the validation error just printed to the Log, and on how tracking felt during the run) before committing to an Autoscroll session with it. This use case is included by Use Case 2.1, 2.2, and 2.3, rather than performed on its own.	
Preconditions:		1. A Calibration or Quick Calibrate run has just finished.	
Postconditions:		1. The including use case proceeds to Autoscroll, repeats the calibration run, or is cancelled back to the Main Page, per the User's choice.	

Priority:	High
Frequency of Use:	Once per Calibration or Quick Calibrate run.
Flow of Events:	1. The system displays a dialog: “Calibration finished -- check the validation error printed above in the log,” offering Yes (launch now), No (redo the calibration), and Cancel (stop here). 2. The User chooses one of the three options. 3a. If Yes, control returns to the including use case to proceed to Autoscroll. 3b. If No, control returns to the including use case to repeat its own calibration step. 3c. If Cancel, control returns to the Main Page without starting Autoscroll.
Alternative Flows:	None.
Exceptions:	None.
Includes:	None.
Special Requirements:	Requirement 3.3.4.
Assumptions:	None.
Notes and Issues:	This replaces the automatic, confidence-score-triggered recalibration prompt described in Lab 1/2 (old Use Case 1.3), which the implementation does not have: there is no continuously-computed, exposed Confidence Score outside the Math Model's own internal prediction blending.

Use Case 2.5 — Clear All Calibrations

Use Case ID:	2.5		
Use Case Name:	Clear All Calibrations		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		The User discards every stored Calibration Profile and session at once, from the Settings page, so that both Gaze Models require a full recalibration before they will run again.	
Preconditions:		1. The application is running.	
Postconditions:		1. The stored Math Model Calibration Profile, the CNN Model's per-user correction, and all stored calibration sessions (used by Quick Start's pooling option) have been deleted.	
Priority:		Low	
Frequency of Use:		Rarely -- typically after a setup change (new desk, new lighting, different chair height)	

	significant enough that old calibration data would actively hurt pooling.
Flow of Events:	1. The User opens the Settings page and selects “Clear all calibrations”. 2. The system asks the User to confirm, restating what will be deleted. 3. The User confirms. 4. The system deletes the Math Model profile, the CNN Model correction, and all stored calibration sessions.
Alternative Flows:	2.5.AC.1: If the User does not confirm, nothing is deleted.
Exceptions:	None.
Includes:	None.
Special Requirements:	Requirement 3.3.5.
Assumptions:	None.
Notes and Issues:	None.

Use Case 3.1 — Estimate Gaze Point

Use Case ID:	3.1		
Use Case Name:	Estimate Gaze Point		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		While an Autoscroll session is running, the system continuously determines the point on screen the User is currently looking at, using whichever Gaze Model (Math or CNN) that session was started with, so the point can feed the Scroll Trigger.	
Preconditions:		1. Camera access has been granted. 2. A Calibration Profile exists for the active Gaze Model.	
Postconditions:		1. A current, smoothed Gaze Point is available to Use Case 4.2 (Evaluate Scroll Trigger), or no Gaze Point is produced for this frame (see Exceptions).	
Priority:		High	
Frequency of Use:		Continuously while an Autoscroll session is running, once per webcam frame.	
Flow of Events:		1. The system detects face landmarks and estimates Head Pose for the current frame. 2. The system checks whether the frame is usable (face detected, eyes not closed). 3. The system computes a raw Gaze Point	

	using the active Gaze Model (Math or CNN). 4. The system clamps the raw Gaze Point to within 50% of the screen size beyond each edge. 5. The system smooths the clamped Gaze Point with a Kalman filter and an exponential moving average. 6. The system makes the smoothed Gaze Point available to Use Case 4.2 (Evaluate Scroll Trigger).
Alternative Flows:	None.
Exceptions:	3.1.EX.1: If the User's face cannot be detected, their eyes are closed (a blink), or (Math Model only) a usable gaze feature cannot be extracted for this frame, the system produces no Gaze Point for that instant and resets the Scroll Trigger's dwell state, so a stale reading cannot cause a scroll.
Includes:	None.
Special Requirements:	Requirement 3.2.1 through Requirement 3.2.6.
Assumptions:	None.
Notes and Issues:	No per-frame Confidence Score is exposed by this use case; see Use Case 2.4's Notes for why the Lab 1/2 recalibration trigger built on one has been removed.

Use Case 4.1 — Scan Screen Content

Use Case ID:	4.1		
Use Case Name:	Scan Screen Content		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		On macOS, the system periodically scans the currently Focused Window for Content Blocks (recognized text and its position) using Apple's Vision OCR framework, on a background thread, so this scan's latency never blocks the main tracking loop's frame rate.	
Preconditions:		1. The system is running on macOS with the Vision-based content scanner available.	
Postconditions:		1. An up-to-date list of Content Blocks for the Focused Window is available to Use Case 4.2 (Evaluate Scroll Trigger).	

Priority:	Medium
Frequency of Use:	Continuously while an Autoscroll session is running, about every 0.4 seconds, independent of the main tracking loop's own frame rate.
Flow of Events:	1. The system determines the current Focused Window's bounds. 2. The system captures and OCR-scans that window's region of the screen. 3. The system reports the resulting Content Blocks for Use Case 4.2 (Evaluate Scroll Trigger) to read on its next tick.
Alternative Flows:	None.
Exceptions:	4.1.EX.1: If the Vision-based content scanner is unavailable -- any platform other than macOS, or a macOS installation without the Vision dependency -- this use case does not run at all for the rest of the session, and Use Case 4.2 falls back to tier 1 (bottom-of-window) only.
Includes:	None.
Special Requirements:	Requirement 3.4.2; Requirement 3.4.4.
Assumptions:	None.
Notes and Issues:	This replaces Lab 1/2's “Identify Important Content Regions”: there is no separate importance classification step -- OCR only ever reports text blocks and positions, and Use Case 4.2 is what decides when enough of them have been read.

Use Case 4.2 — Evaluate Scroll Trigger

Use Case ID:	4.2		
Use Case Name:	Evaluate Scroll Trigger		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		On every frame with a usable Gaze Point, the system decides whether to scroll the Focused Window, using the first of three tiers that applies: a quick bottom-of-window check, an OCR-based “reached the end of visible text” check, or a broader, more conservative “nothing unseen ahead” check.	
Preconditions:		1. A current Gaze Point is available (Use Case 3.1).	

	2. The Focused Window's bounds are known.
Postconditions:	1. A scroll decision (fire or not) has been made for this frame, and, if it fired, Use Case 5.1 (Autoscroll Page) has scrolled the window.
Priority:	High
Frequency of Use:	Continuously while an Autoscroll session is running, once per frame with a usable Gaze Point.
Flow of Events:	1. The system expresses the Gaze Point as a fraction of the Focused Window's own height. 2. Tier 1: if that fraction has dwelt in the bottom 15% of the window for a short period, the system decides to scroll. 3. Tier 2 (if tier 1 did not fire and Content Blocks are available): if the Gaze Point has reached and moved past the end of the lowest visible Content Block, the system decides to scroll. 4. Tier 3 (if tiers 1-2 did not fire and Content Blocks are available): if nothing unseen remains between the Gaze Point and the bottom of the window, after a longer dwell than tier 1, the system decides to scroll. 5. If none of the tiers fire, the system makes no scroll decision for this frame.
Alternative Flows:	4.2.AC.1: If Content Blocks are unavailable for this session (Use Case 4.1's Exception), only tier 1 is ever evaluated.
Exceptions:	None.
Includes:	Use Case 3.1 (Estimate Gaze Point); Use Case 4.1 (Scan Screen Content, where available).
Special Requirements:	Requirement 3.4.1; Requirement 3.4.3; Requirement 3.4.4; Requirement 3.4.5.
Assumptions:	None.
Notes and Issues:	The dwell tracking behind tiers 2-3 is deliberately tolerant of calibration noise (a configurable hit-test distance and a short grace period before a momentary divergence resets progress), since real calibration error is large relative to a line of text's height.

Use Case 5.1 — Autoscroll Page

Use Case ID:	5.1
Use Case Name:	Autoscroll Page

Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		The system scrolls the Focused Window downward, automatically, whenever Use Case 4.2 (Evaluate Scroll Trigger) decides to fire, so the User can keep reading without scrolling by hand.	
Preconditions:		1. Calibration is complete for the active Gaze Model (Use Case 2.1, 2.2, or 2.3). 2. Autoscroll is not currently paused (Use Case 5.2).	
Postconditions:		1. The Focused Window has scrolled downward by a fixed step, or remains unchanged if the Scroll Trigger did not fire this frame.	
Priority:		High	
Frequency of Use:		Continuously while an Autoscroll session is running and not paused.	
Flow of Events:		1. The system obtains a scroll decision from Use Case 4.2 (Evaluate Scroll Trigger). 2. If the decision is to fire, the system briefly moves the cursor to the Focused Window's center and posts a native scroll-wheel event scrolling it downward by a fixed step, then restores the cursor's position. 3. The system continues monitoring the Gaze Point for the next frame.	
Alternative Flows:		5.1.AC.1: If the Attention Gate pauses scrolling at any point during this flow, see Use Case 5.2 (Pause Autoscroll on Look-Away). 5.1.AC.2: The User may end the session at any time; see Use Case 5.3 (End Autoscroll Session).	
Exceptions:		None.	
Includes:		Use Case 3.1 (Estimate Gaze Point); Use Case 4.2 (Evaluate Scroll Trigger).	
Special Requirements:		Requirement 3.5.1.	
Assumptions:		None.	
Notes and Issues:		The system only ever scrolls downward, by the same fixed step each time; there is no up/left/right scrolling, and no User-facing sensitivity or speed setting (see srs.docx Section 6, Known Gaps).	

Use Case 5.2 — Pause Autoscroll on Look-Away

Use Case ID:	5.2		
Use Case Name:	Pause Autoscroll on Look-Away		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		The system automatically pauses all scrolling, with no action from the User, whenever a coarse check on their Head Pose shows they have turned or tilted away from the screen, and resumes automatically once they turn back.	
Preconditions:		1. An Autoscroll session is running.	
Postconditions:		1. Autoscroll is paused while the User's head is outside the configured angle range, and resumes automatically once it returns within range.	
Priority:		Medium	
Frequency of Use:		As often as the User naturally looks away from the screen during a session (e.g. to glance at a second monitor or speak to someone).	
Flow of Events:		1. The system evaluates the current Head Pose's yaw and pitch against configured angle limits on every frame. 2. If the angle has been outside the limits for a short grace period, the system pauses Autoscroll and resets the Scroll Trigger's dwell state. 3. The system continues evaluating Head Pose while paused. 4. Once the Head Pose returns within the angle limits, the system resumes Autoscroll automatically, with no prompt or confirmation from the User.	
Alternative Flows:		None.	
Exceptions:		None.	
Includes:		None.	
Special Requirements:		Requirement 3.5.2.	
Assumptions:		None.	
Notes and Issues:		This replaces Lab 1/2's keyboard-shortcut / on-screen pause-resume control (old Use Case 5.2 and old Requirement 3.5.4), which does not exist in the implementation: there is no manual pause, only this automatic look-away gate.	

Use Case 5.3 — End Autoscroll Session

Use Case ID:	5.3		
Use Case Name:	End Autoscroll Session		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		The User ends a running Autoscroll session at any time, returning to the Main Page.	
Preconditions:		1. An Autoscroll session is running.	
Postconditions:		1. The session has ended, the webcam has been released, and the Main Page's buttons are re-enabled.	
Priority:		Medium	
Frequency of Use:		Once per Autoscroll session, whenever the User is done reading.	
Flow of Events:		1. The User presses 'q' in the session's window. 2. The system releases the webcam and closes the window. 3. The system returns to the Main Page.	
Alternative Flows:		None.	
Exceptions:		None.	
Includes:		None.	
Special Requirements:		Requirement 3.5.3.	
Assumptions:		None.	
Notes and Issues:		This is the only way to stop a session; there is no separate pause control the User can trigger themselves (see Use Case 5.2).	

Use Case 6.1 — Configure Appearance

Use Case ID:	6.1		
Use Case Name:	Configure Appearance		
Created By:	Project Team	Last Updated By:	Project Team
Date Created:		6 September 2026	
Date Last Updated:		6 November 2026	
Actor:		User	
Description:		The User switches the launcher window's own Light or Dark appearance from the Settings page.	
Preconditions:		1. The application is running.	
Postconditions:		1. The launcher's colours update immediately to the chosen appearance, for the current	

	launch only.
Priority:	Low
Frequency of Use:	Occasionally, typically once per session if the default (matching the operating system's current appearance) is not to the User's liking that session.
Flow of Events:	1. The User opens the Settings page. 2. The User selects Light or Dark. 3. The system re-applies its colour palette to every page immediately.
Alternative Flows:	None.
Exceptions:	None.
Includes:	None.
Special Requirements:	Requirement 3.6.2.
Assumptions:	None.
Notes and Issues:	This choice is not persisted as an override: the next launch starts from whatever the operating system's own appearance setting is at that time.